Card 1 · The seven steps

Same flow for every task. Always start with step 1.

#	Step	Do this	You have…
1	**Run it**	Build `-O0 -g`. Run. Save command, input, output. Run twice and `diff`.	The exact line that looks wrong
2	**Understand it**	Purpose, premise, desired state, boundary, data path. Declarations first.	A pass on **Gate 1**
3	**State it**	Fill the Behavior Card: Declare, Define, Implement. Mark unknowns `?`.	A pass on **Gate 2**
4	**Probe it**	One question per probe. **Write the prediction first.**	Probe output
5	**Pin it**	First bad state, or every change point. One sentence.	"X is Y because line L … when C"
6	**Prove it**	Test the sentence by hand (`set var`). Smallest change. Re-run the same probes.	**Gate 3**
7	**Log it**	Journal, hypothesis ledger, save probe files.	Someone else could repeat it

GATE **Gate 1 · Confidence. Explain** it in 3 plain sentences. **Predict** the next two output lines, then run. **Locate** the module, one datum, one function.

GATE **Gate 2 · Card.** Every DEFINE line is a testable expression. Every datum has a name and a location. Every `?` has a probe.

GATE **Gate 3 · Proof.** The probe **fired** on the old build and is **silent** on the new one (read its hit count). Old behavior unchanged. Something stays behind: an assert, a test, a saved probe.

A debugger answers: *at this **place**, when this **condition** holds, what is this **data**?* You supply all three.

Card 2 · Behavior Card

BEHAVIOR CARD

Task: _____ **Date:** _____ **Type:** A□ B□ C□ ___

INTAKE type · first probe · time · unknowns · check-in

RUN command and the exact line you saw

EXPECTED and who says so (ticket, spec, person)

PURPOSE one sentence

DECLARE — what exists (name · type · file:line · which instance)

DEFINE — what must be true

P before / trigger

Q after / expected

I always true

Q′ observed instead

IMPLEMENT — what changes it writers · readers · deciding branch

? unknowns, each with its probe

PROBES verb · place · condition · *prediction* · if wrong

HYPOTHESES H1

H2

Card 3 · Intake: classify in five minutes

Ask: What is being asked? How sure is the *expected* behavior? How many `?` on the card?

Family	Type	Sounds like…	First probe
A explain	A1	Who calls this? Only here?	LOG callers
	A2	How does this module work?	TRAP + LOG state writes
	A3	What variable controls this?	TRAP the output, walk back
	A4	Where does the data go?	Tag the value, FILTER per stage
B fix	B1	Wrong value	FILTER at the place
	B2	Didn't happen	Probe the **decision**, not the action
	B3	Happened, shouldn't	Break at action `if !(expected)`
	B4	Wrong order / timing	LOG both with sequence numbers
	B5	Stuck	Ctrl-C, `bt`, tripwire on a counter
	B6	Crash	Stop at signal, `up`, trace the bad value
	B7	Only sometimes	Make deterministic; diff bad vs good
	B8	Slow / growing	**Measure first** (profiler)
C build	C1	When X, do Y	LOG every way X becomes true
	C2	New data	Find the set / read / reset points
	C3	Change a rule	Find guard and **all** uses; golden log
	C4	New event	Follow a similar event (A4)
	C5	Connect to a module	Write the contract; LOG the boundary
	C6	Refactor	Golden log; `diff` after each step

Intake line: `TYPE · first probe · time · unknowns · check-in at …`

Unknowns (a heuristic): 0–2 go · 3–5 add to step 2 · 6+ split or clarify first. **Mixed task?** Do the A part first.

Card 4 · Time and the stuck clock

Starting budgets: this book's proposal, not a research result. Calibrate with your journal.

Step	A explain	B fix	C build	4-hour task: A / B / C (min)
1 Run	10%	10%	5%	24 / 24 / 12
2 Understand	30%	20%	25%	72 / 48 / 60
3 State	20%	15%	15%	48 / 36 / 36
4 Probe	25%	25%	15%	60 / 60 / 36
5 Pin	5%	10%	10%	12 / 24 / 24
6 Prove (incl. change)	5%	15%	25%	12 / 36 / 60
7 Log	5%	5%	5%	12 / 12 / 12

Gates by: Gate 1 at A 40% · B 30% · C 30%. Gate 2 at A 60% · B 45% · C 45%. Gate 3 before the log step.

> **RULE** **Any step over 1.5× its budget: stop.** Run the reset below, then **ask within 15 minutes**.

Four-question reset

1. Is the **expected** behavior written down, with a source?
2. Is the **data** on my card the data that matters? (Change it by hand: does the symptom respond?)
3. Can I make the problem **smaller**? (fewer events, one instance, a shorter run)
4. What would I **ask**, and what is my best **hypothesis**? Write both.

My plan: T = _____ Check-in at _____ Escalate to _____

Card 5 · English → condition

`END` = last line of the function that changes the data.

The English says…	Logic	Probe
Always / never	`I` holds everywhere	`break END if !(I)`
After X, Y holds	`{P} X {Q}`	at X's exit `if !(Q)`; or `finish`
When X do Y	`if (G) A`	break **at A**; and at the test of G, show operands
Y **didn't happen**	G false, or never tested	break at the **decision**; count hits of the function
Y happened, **shouldn't**	A ran while G false	break at A `if !(G_expected)`; `bt`
Only if X	`P` before `C`	at C entry `if !(P)`
The **Nth time**	counter	ignore count, or `$n` + condition
In this order	S1 then S2	log both with a number; or **arm**
Stuck	progress var must change	Ctrl-C + `bt`; tripwire; silent watchpoint
Only this one instance	scope	add `this == $target` / `thread N` / `$_streq`

Say it aloud. If the condition does not read as the behavior, rewrite it.

Place: invariant → end of the writer · postcondition → exit · didn't happen → the decision · data must be in scope.

> **TRAP** **A silent probe proves nothing** until you have seen it fire. Loosen it, or run it on a known-bad case.

Card 6 · Probe chooser

Verb	Does	Answers	gdb	lldb
FILTER	stop if true	the first bad moment	`break L if C`	`breakpoint set … -c 'C'`
COUNT	tally, no stop	how often?	`set $n=$n+1` in `commands`	callback → False
LOG	print, keep going	values over time	`dprintf L,"fmt",x`	callback → False
TRAP	stop on change	who writes this?	`watch -l x`	`watchpoint set variable x`
ARM	one enables another	only after X	`disable N` / `enable N`	callback
REWIND	go back	who made it bad?	`reverse-continue` (rr)	—

You know…	Use
a place; data looks wrong there	**FILTER**
a place that runs 1000s of times	ignore count, or COUNT
a place; don't want to stop	**LOG**
a **datum**, not a place	**TRAP** (`watch -l`)
only bad values matter	TRAP with `if`
who **reads** it	`rwatch` / `awatch`
rare, only **after** an event	**ARM**
an event class	`catch throw` / `signal` / `syscall`
bad **now**, cause earlier	**REWIND**, or restart with a TRAP
a crash you can't repro	core dump
a **guess** about the cause	`set var`, `continue` (no edit)

One question per probe. Write the prediction first.

Card 7 · Recipes (fill in the CAPS)

```
# FILTER   stop the first time the rule is broken
break FILE:LINE if COND

# LOG      print without stopping
dprintf FILE:LINE,"t=%d x=%d\n", TICK, X

# COUNT    tally, report at the end
set $n = 0
break FUNC
commands
  silent
  set $n = $n + 1
  continue
end

# TRAP     who writes this? (log each writer, keep running)
break FUNC_WHERE_OBJECT_IS_VISIBLE
run ARGS
watch -l OBJ.FIELD
commands
  silent
  bt 3
  continue
end

# TRAP+IF  only bad values
watch -l OBJ.FIELD if OBJ.FIELD > LIMIT

# ARM      watch only after a trigger
watch -l OBJ.FIELD
disable N                     # N = that watchpoint's number
break TRIGGER_FUNC if COND
commands
  silent
  enable N
  continue
end

# INSTANCE one object only
set $target = this            # while stopped in the object you want
condition N this == $target

# EXPERIMENT  test a hypothesis before editing
set var FIELD = VALUE
continue

# LOOK       whole object, caller, return value
print *this        up        print VAR        finish
```

Run a saved probe: `gdb -q -batch -x probe.gdb ./prog` (lldb: `lldb -b -s probe.lldb ./prog`)

Card 8 · Pin it

Goal: one sentence naming the exact line **and instance**.

> *is because when .*

Five moves (bug)

1. **FILTER** for the **first** bad observation (from the card).
2. **Bad on entry, or made inside?** `info args`, `print *this`, then `up`.
3. Bad on entry → **go up one stage**, repeat 1.
4. Made inside → **TRAP the writers** (`watch -l`), compare to the card.
5. **Name it. Test it:** `set var`, `continue`. Symptom changes? Then edit.

Which instance?

1. Pick by **state**, not name: stop *in* the bad object. `this` is it.
2. **Record it:** `set $target = this` (lldb: copy the address from `p this`).
3. **Scope** the next probe: `condition N this == $target`, or `watch -l`.
4. Across a **message boundary**: tag the value; LOG at send and receive.
5. Addresses change each run: re-find with a stable field.

For a feature (find every change point)

file:line	what happens	seen in run?	change needed?	why

Runtime shows what ran. Reading shows what exists. Every difference is a path to reason about.

Card 9 · Understand checklist

Five questions (one or two lines each)

1. **Purpose.** Why does this exist? Who uses its output?
2. **Premise.** What must be true for it to work? Inputs, assumptions.
3. **Desired state.** What does “working” look like, observably?
4. **Boundary.** What is mine? What is a black box (and what do I assume about it)?
5. **Data path.** Where does it enter, change, leave?

Read in this order: the run · ticket or spec · **declarations** (headers, data members) · tests · diagrams · history (`git log -S name`) · a person (with a hypothesis).

> **RULE** **Slice rule.** Read completely everything that reads or writes the data on your card (one hop). Everything else: a black box with a written assumption. Stop when the card can be filled.

Shared-state check (modules with no messages can still interact): globals · queues and buffers · files and config · hardware registers · timing · locks. Or watch your data in a full run and see who writes it.

If no diagrams, draw three: (1) context boxes · (2) one value’s data path · (3) **state × event table** from the *requirements*.

state event	e1	e2	e3
s1			
s2			

> **GATE** **Gate 1.** Explain (3 sentences) · Predict (next 2 lines) · Locate (module, datum, function).

Card 10 · Prove it, log it

Proof: seven checks

1. The sentence was **tested first** (`set var`).
2. **Smallest** change; one thing at a time.
3. **Baseline run** re-run; `diff` the output.
4. **Same probes silent**; read the **hit count**.
5. The probe was **seen to fire** on the old build.
6. **Neighbors unchanged** (tally before and after; `diff`).
7. **Card → code**: `assert` or a test of the invariant.

Feature proof: new behavior on **every** path in the change-point table; old behavior byte-for-byte unchanged.

JOURNAL

Task / type:

Cause or change (one sentence)

Fix or feature file:line, commit

Proof what you ran, what you saw

Saved probes, tests

First wrong assumption

Planned / actual minutes per step 1 / 2 / 3 / 4 / 5 / 6 / 7 /

Hypothesis ledger: H# · hypothesis · prediction · probe · result (confirmed / rejected).

Weekly, five minutes: read the “first wrong assumption” lines. What repeats?